

# Four questions about how the book is actually shaped

Answers to exactly what you asked, in order, each with the picture that makes it click.

## 1 Is it a map to a list of PriceLevels?

"is it price keys to a value which is a vector/array/list of pricelevels?"

### SHORT ANSWER

No — one price, one `PriceLevel`. The *list* lives one layer deeper, **inside** that single `PriceLevel`, not as the map's value type itself.

**OrderBook's three members, then one entry zoomed in**

FIG. 01

**bidLevels : map<Price, PriceLevel>**

64840.09 → PriceLevel

64840.10 → PriceLevel

64840.11 → PriceLevel

**askLevels : map<Price, PriceLevel>**

64840.12 → PriceLevel

64840.13 → PriceLevel

64841.00 → PriceLevel

↓ one key, zoomed in – the value at 64840.11 is exactly one PriceLevel object ↓

### PriceLevel (the value stored at key 64840.11)

this one object represents everything resting at this one price

total\_quantity (cached, kept up to date – see Fig. 02) **1.60 BTC**

FIFO list:

order 101  
0.50 BTC

→

order 102  
0.75 BTC

→

order 205  
0.35 BTC

**locators : unordered\_map<OrderId, OrderLocator>**

id 101 → side=buy, price=64840.11, position 1st

id 102 → side=buy, price=64840.11, position 2nd

id 205 → side=buy, price=64840.11, position 3rd

each locator points into the exact same three FIFO slots shown above – it's a second index into the same orders, not a copy of them

Same reasoning for the locator map: one `OrderId` key, one `OrderLocator` value — never a list, because an id can only belong to one order at a time (that's the "no duplicate id" rule from ADR 0012). `Price` works as a map key at all because it already has `operator<=>` defined, from back when it was first built — that's the exact thing `std::map` needs to keep keys ordered.

---

## 2 What does "a running total quantity" mean?



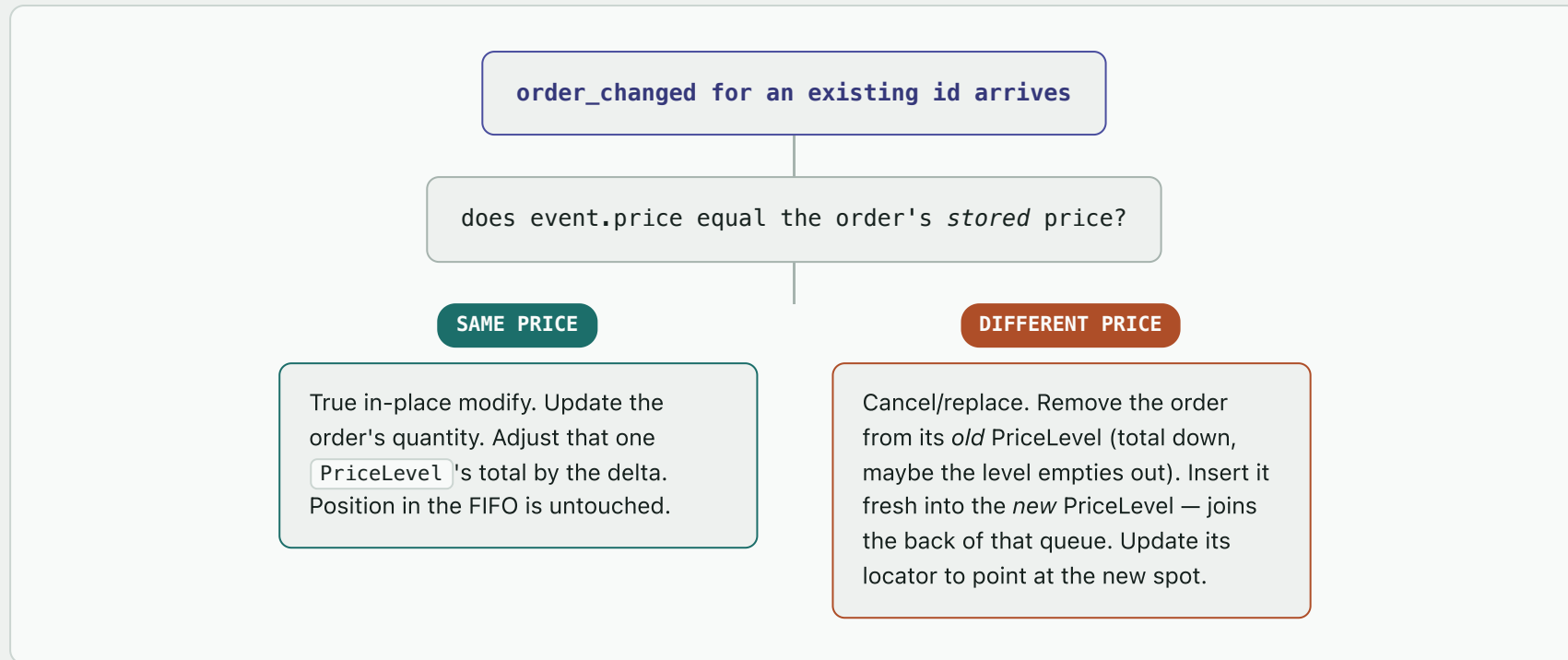
This is a general pattern, not specific to order books — the same idea as a bank keeping a running balance instead of re-adding every transaction each time you check it.

### 3 Modify: resolving the ADR 0012 conflict

ADR 0012 says a modify's price must match what's stored, or it's rejected. The audit script's real-data finding says Bitstamp `order_changed` can genuinely move price. Both can't be right as written — here's the resolution, and it has a real, named industry answer.

#### THE NAMED CONCEPT: CANCEL/REPLACE

FIX protocol (the industry-standard exchange messaging protocol) has an explicit message for this — `OrderCancelReplaceRequest` (tag 35=G) — which lets a trader change an order's price and/or quantity while keeping the same reference id for convenience. The rule every price-time-priority venue follows is universal: **changing price always forfeits queue position**; only a same-price quantity *decrease* preserves it. This is the same rule from a few turns back about size increases going to the back of the queue — a price change is the more extreme version of the same idea.

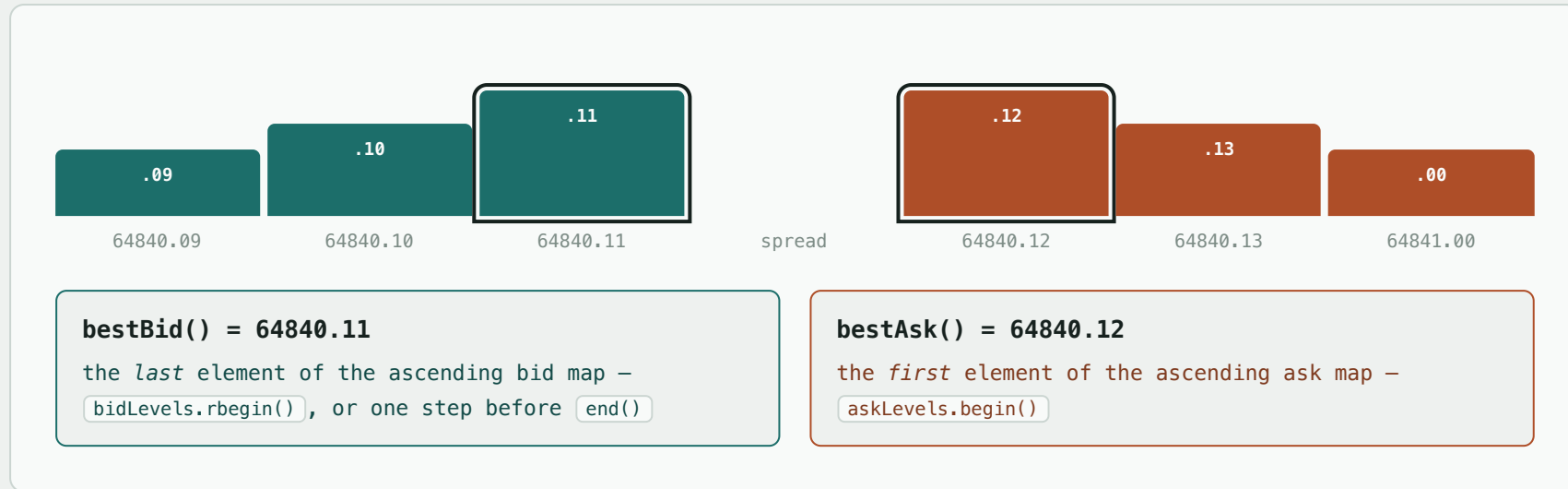


This isn't picking a side arbitrarily — it's what the data already told the project (real `order_changed` messages that move price) combined with how every real venue actually treats a price-changing modify. ADR 0012's rule text is the thing to update, not the data.

#### 4 Why the map's own order gives you best bid/ask for free

## Ascending order, both sides — the spread is the empty middle

FIG. 04



Both maps are sorted ascending, always — that's `std::map`'s whole nature. "Best" means opposite ends for the two sides: for bids, higher is better, so the best one ends up sorted *last*. For asks, lower is better, so the best one ends up sorted *first*. Neither map needs its own "what's the best price" tracking — the sort order already *is* that answer.

---

Real values above (64840.11 / 64840.12) match the reference snapshot's actual best bid/ask, for continuity with earlier work.